

# 실습: 사전 지도 기반 위치 추정

AMCL with Gazebo

[cchyun@gmail.com](mailto:cchyun@gmail.com)

# Part 1. 전체 시스템 구조

- 로봇이 자신의 위치를 알기 위해 데이터가 어떻게 흐르는지 이해하는 것이 첫 단계
  - 입력 데이터
    - `map_server` → `/map` : Day 11에서 저장한 정적 지도(Static Map) 정보를 AMCL에 제공
    - LIDAR 센서 → `/scan` : 현재 로봇 주변의 장애물 거리 데이터를 AMCL에 제공
    - Odometry → `odom → base_link` TF : 바퀴 회전수·IMU 기반 상대적 이동 거리를 제공
  - AMCL 노드 (연산)
    - 지도와 센서 데이터를 비교하여 로봇의 가장 확률 높은 위치를 계산

- AMCL이 계산한 위치 정보는 세 가지 형태로 출력
  - `map → odom` TF
    - 오도메트리의 오차를 보정하여 지도상에서의 정확한 좌표 변환 값을 발행
  - `/amcl_pose`
    - 로봇의 현재 추정 위치와 불확실성(Covariance) 데이터
  - `/particle_cloud`
    - 로봇이 존재할 가능성이 있는 후보지들을 나타내는 파티클 무리

## Part 2. 단계별 구현

## Step 1 — Gazebo World 기동

- SLAM 없이 로봇의 물리 시뮬레이션 환경만 구동
  - 10m×10m 공간에 외곽 벽, 내부 칸막이, 다양한 박스·원통 장애물이 배치된 가상 세계
  - 로봇과 가상 세계를 띄우는 첫 번째 단계
  - `use_sim_time:=True`가 기본 적용됨

*# 터미널 1: 시뮬레이션 환경 실행*

```
cd ~/Workspace/ros2-ws
source install/setup.bash
ros2 launch my_robot_description amcl.launch.py
```

## Step 2 — 지도 서버(map\_server) 실행

- 저장된 지도를 불러와 ROS 네트워크에 배포
  - Day 11에서 저장한 지도 파일 경로를 YAML로 지정
  - **오류 주의:** 지도가 로드되어도 `lifecycle` 노드이므로 아직 활성화되지 않은 상태
  - `map_server`는 지도 이미지만 발행하고 `map` frame 자체는 만들지 않습니다

```
# 터미널 2: map frame 발행
ros2 run tf2_ros static_transform_publisher 0 0 0 0 0 0 map odom
```

```
# 터미널 3: 지도 서버 실행 (Day 11에서 저장한 절대 경로 입력)
cd ~/Workspace/ros2-ws
source install/setup.bash
ros2 run nav2_map_server map_server \
  --ros-args \
  -p yaml_filename:=/home/cchyun/Workspace/ros2_ws/room_map.yaml \
  -p use_sim_time:=True
```

- 파라미터가 적용된 위치 추정 노드 실행

- `min_particles` (최소 파티클 수), `max_particles` (최대 파티클 수) 등이 YAML에 포함되어야 함
- 파라미터 YAML 파일 경로를 반드시 함께 지정



## Step 3 — my\_robot\_description/config/my\_amcl\_params.yaml

```
amcl:
  ros__parameters:
    # 1. 파티클 설정
    min_particles: 500
    max_particles: 2000
    pf_err: 0.05
    pf_z: 0.99
    recovery_alpha_slow: 0.001
    recovery_alpha_fast: 0.1

    # 2. 모션 모델 설정 (차동 구동형)
    robot_model_type: "nav2_amcl::DifferentialMotionModel"
    alpha1: 0.2
    alpha2: 0.2
    alpha3: 0.2
    alpha4: 0.2
```

## Step 3 — my\_robot\_description/config/my\_amcl\_params.yaml

```
# 3. 센서 모델 설정
laser_model_type: "likelihood_field"
laser_min_range: -1.0
laser_max_range: -1.0
laser_likelihood_max_dist: 2.0
z_hit: 0.5
z_rand: 0.5
z_short: 0.05
z_max: 0.05

# 4. 업데이트 주기
update_min_d: 0.25
update_min_a: 0.2

# 5. 프레임 설정
base_frame_id: "base_footprint"
odom_frame_id: "odom"
global_frame_id: "map"
tf_broadcast: true
```

## Step 3 — AMCL 노드 실행

```
# 터미널 4: AMCL 실행 (파라미터 YAML 파일 경로 포함)
cd ~/Workspace/ros2-ws
source install/setup.bash
ros2 run nav2_amcl amcl \
  --ros-args \
  --params-file src/my_robot_description/config/amcl_params.yaml \
  -p use_sim_time:=True
```

## Step 4 — Lifecycle Manager 활성화

- Nav2 노드들은 순서에 맞춰 활성화해주어야 함
  - `map_server`와 `amcl` 순서로 지정
  - `autostart:=True` 옵션으로 자동 활성화
  - 실행 결과: AMCL이 지도를 받고 센서 데이터를 기다리는 상태가 됨
  - 이 시점부터 `map_server`가 `/map` 토픽을 발행합니다

```
# 터미널 5: 노드 상태 관리자 실행
cd ~/Workspace/ros2-ws
source install/setup.bash
ros2 run nav2_lifecycle_manager lifecycle_manager \
  --ros-args \
  -p node_names:="['map_server', 'amcl']" \
  -p autostart:=True \
  -p use_sim_time:=True
```

## Step 5 — RViz2 설정

- 로봇의 위치를 시각적으로 확인하고 초기 위치를 잡아줌

- RViz2 실행

```
sudo apt install ros-humble-nav2-rviz-plugins  
rviz2
```

- 레이어 추가 (Add → By topic 탭 선택)
  - Map (Topic: /map) — Durability Policy: Transient Local
  - ParticleCloud (Topic: /particle\_cloud) — Nav2 플러그인 필요
  - LaserScan (Topic: /scan)
  - TF

## Step 5 — 초기 위치 설정 (/initialpose 토픽 발행)

- 터미널에서 초기 위치를 발행

```
# 터미널 6: 초기 위치 발행
cd ~/Workspace/ros2-ws
source install/setup.bash
ros2 topic pub --once /initialpose \
  geometry_msgs/msg/PoseWithCovarianceStamped \
  "{header: {frame_id: 'map'}, pose: {pose: {position: \
{x: 0.0, y: 0.0, z: 0.0}, orientation: {w: 1.0}}}}"
```

## Step 6 — Teleop 주행 및 수렴 확인

- 로봇을 움직여 파티클이 어떻게 변화하는지 관찰
  - 로봇이 움직이면서 센서 데이터가 업데이트되면 파티클이 수렴(Convergence)

```
# 터미널 7: 키보드 제어
cd ~/Workspace/ros2-ws
source install/setup.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

- 관찰 포인트
  - 초기: 파티클이 설정한 위치 주변에 넓게 분포
  - 주행 후: 파티클들이 로봇의 실제 위치로 점점 **좁게 수렴**
  - 파티클이 작고 촘촘한 구름 형태가 되면 위치 추정이 안정화된 것

- 로봇이 어디 있는지 모르는 '미아 상태'에서 위치를 찾는 실험

- 1단계: 서비스 호출

```
# 터미널 8: 글로벌 로컬라이제이션 재초기화  
cd ~/Workspace/ros2-ws  
source install/setup.bash  
ros2 service call /reinitialize_global_localization std_srvs/srv/Empty {}
```

- 2단계: 관찰

- 파티클이 지도 전체에 고르게 뿌려짐

- 3단계: 해결

- 로봇을 제자리에서 천천히 회전시키거나 주행하면 파티클이 특정 지점으로 다시 모임



## Step 8 — 수렴 그래프 확인 (Data Analysis)

- 위치 추정의 정확도(공분산)를 그래프로 분석

- X, Y축 방향의 오차와 회전 오차가 주행함에 따라 줄어드는 것을 시각적으로 확인

```
# 터미널 9: 공분산 그래프 출력
```

```
cd ~/Workspace/ros2-ws
```

```
source install/setup.bash
```

```
ros2 run rqt_plot rqt_plot /amcl_pose/pose/covariance[0] /amcl_pose/pose/covariance[7] /amcl_pose/pose/covariance[35]
```

- 관찰 포인트

- 주행 전: 공분산 값이 높음 (위치 불확실)
- 주행 후: 공분산 값이 낮아짐 (위치 확정)
- 그래프가 낮은 값에서 안정적으로 유지되면 로컬라이제이션 성공

- 위 단계들을 하나의 Launch 파일로 통합하여 실행할 수도 있음

- `my_robot_description` 패키지의 `amcl.launch.py`

```
cd ~/Workspace/ros2-ws
source install/setup.bash
ros2 launch my_robot_description amcl.launch.py \
  world:=room_world.world \
  map_yaml:=/home/cchyun/Workspace/ros2_ws/room_map.yaml
```

## **Part 3. 자주 발생하는 오류와 해결법**

```
ros2 topic list           # 현재 발행 중인 토픽 목록
ros2 topic echo /amcl_pose --once # AMCL 추정 위치 확인
ros2 topic hz /scan       # LiDAR 데이터 수신 주기 확인
ros2 node list            # 현재 동작 중인 노드 확인
ros2 run tf2_tools view_frames # TF 트리 시각화 (PDF 생성)
```

## **Part 4. 심화 미션**

- 복잡한 맵에서 비교 실험

```
cd ~/Workspace/ros2-ws
source install/setup.bash
ros2 launch my_robot_description amcl.launch.py \
world:=slam_world.world \
map_yaml:=/home/cchyun/Workspace/ros2_ws/slam_map.yaml
```

## 심화 미션 2 — 수렴 감시 노드 작성 (Python)

- `/amcl_pose` 토픽 구독 → 공분산이 임계값 이하로 떨어지면 성공 로그 출력
  - `/amcl_pose` 토픽을 구독
  - `pose.covariance[0]` (X), `pose.covariance[7]` (Y) 값이 `0.05` 이하로 떨어지면 로그 출력

```
cd ~/Workspace/ros2-ws/src
# 생성 후 추가
ros2 pkg create my_robot_tools --build-type ament_python --dependencies rclpy std_msgs geometry_msgs sensor_msgs nav_msgs
```

## 심화 미션 2 — 수렴 감시 노드 작성 (Python)

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseWithCovarianceStamped

class ConvergenceMonitor(Node):
    def __init__(self):
        super().__init__('convergence_monitor')
        self.subscription = self.create_subscription(
            PoseWithCovarianceStamped, '/amcl_pose',
            self.pose_callback, 10)
        self.threshold = 0.05

    def pose_callback(self, msg):
        cov_x = msg.pose.covariance[0]
        cov_y = msg.pose.covariance[7]
        if cov_x < self.threshold and cov_y < self.threshold:
            self.get_logger().info('Localization Success! Converged.')

def main(args=None):
    rclpy.init(args=args)
    node = ConvergenceMonitor()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```



- 공분산이 너무 커지면 자동으로 `/global_localization` 서비스를 호출하는 노드

## 심화 미션 3 — 자동 재로컬라이제이션 감시 노드

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseWithCovarianceStamped
from std_srvs.srv import Empty

class LocalizationMonitor(Node):
    def __init__(self):
        super().__init__('localization_monitor')
        self.subscription = self.create_subscription(
            PoseWithCovarianceStamped, '/amcl_pose',
            self.pose_callback, 10)
        self.client = self.create_client(Empty, '/reinitialize_global_localization')
        self.threshold = 0.5
        self.cooldown_sec = 30.0
        self.last_call_time = self.get_clock().now()

    def pose_callback(self, msg):
        cov_x = msg.pose.covariance[0]
        cov_y = msg.pose.covariance[7]
        if cov_x > self.threshold or cov_y > self.threshold:
            now = self.get_clock().now()
            elapsed = (now - self.last_call_time).nanoseconds / 1e9
            if elapsed > self.cooldown_sec:
                self.get_logger().warn(
                    f'High covariance detected ({cov_x:.2f}). Triggering global localization!')
                self.trigger_global_localization()
                self.last_call_time = now
```



```
def trigger_global_localization(self):
    if not self.client.service_is_ready():
        return
    request = Empty.Request()
    self.client.call_async(request)

def main(args=None):
    rclpy.init(args=args)
    node = LocalizationMonitor()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```